

1 Solution — GIAM 1.4

Problem 7

1.1 Problem

Taking binary as the “standard” base, each octal digit corresponds to a binary *triple* and each hexadecimal digit to a binary *4-tuple*. Complete the octal-to-binary and hexadecimal-to-binary tables. (As a check, the 4-tuple for *A* should be 1010 — nicely, *A* is really 10, so reading "1010" as “ten-ten” aids the memory.)

1.2 The Completed Tables

Each entry is simply the digit’s value *written in binary* and padded with leading zeros to a fixed width — 3 bits for octal (since $8 = 2^3$), 4 bits for hexadecimal (since $16 = 2^4$).

Octal — 3 bits each:

octal	0	1	2	3	4	5	6	7
binary	000	001	010	011	100	101	110	111

Table 1.1

Hexadecimal — 4 bits each:

hex	0	1	2	3	4	5	6	7
binary	0000	0001	0010	0011	0100	0101	0110	0111

Table 1.2

hex	8	9	A	B	C	D	E	F
binary	1000	1001	1010	1011	1100	1101	1110	1111

Table 1.3

The check holds: $A \rightarrow 1010$. (And $F \rightarrow 1111$: fifteen is “all ones.”)

1.3 The Tables, Laid Out Together

Each octal digit = 3 bits, each hex digit = 4 bits — just count in binary, zero-padded.

octal $\rightarrow$ 3 bits		hexadecimal $\rightarrow$ 4 bits	
octal	4 2 1	hex	8 4 2 1
0	0 0 0	0	0 0 0 0
1	0 0 1	1	0 0 0 1
2	0 1 0	2	0 0 1 0
3	0 1 1	3	0 0 1 1
4	1 0 0	4	0 1 0 0
5	1 0 1	5	0 1 0 1
6	1 1 0	6	0 1 1 0
7	1 1 1	7	0 1 1 1
		8	1 0 0 0
		9	1 0 0 1
		A	1 0 1 0
		B	1 0 1 1
		C	1 1 0 0
		D	1 1 0 1
		E	1 1 1 0
		F	1 1 1 1

"ten-ten"

Hex 0–7 = a leading 0 plus the octal triple; hex 8–F = a leading 1 plus the octal triple.

Figure 1.1: Left: octal digits 0–7 with their 3-bit binary forms (place values 4 2 1). Right: hex digits 0–F with their 4-bit binary forms (place values 8 4 2 1), the row $A = 1010$ highlighted with the ‘ten-ten’ note. Both columns are just binary counting, zero-padded.

1.4 Remark — Why It’s Just Zero-Padded Binary Counting

There is no real “lookup” to memorise — each row is the digit’s plain binary expansion, given a fixed width. Reading the bit columns as place values makes this transparent:

- **Octal**, bits worth 4, 2, 1: e.g. $5 = 4 + 1 \rightarrow 101$, $6 = 4 + 2 \rightarrow 110$.
- **Hex**, bits worth 8, 4, 2, 1: e.g. $A = 10 = 8 + 2 \rightarrow 1010$, $D = 13 = 8 + 4 + 1 \rightarrow 1101$, $F = 15 = 8 + 4 + 2 + 1 \rightarrow 1111$.

The widths **3** and **4** are forced: octal needs exactly enough bits to count to $7 = 2^3 - 1$, and hex exactly enough to count to $15 = 2^4 - 1$.

1.5 Remark — The Hex Table Contains the Octal Table Twice

Notice the relationship between the two tables. The first eight hex rows (**0–7**) are each just the octal triple with a **leading 0**:

$$\text{hex } 5 = \underbrace{0}_{<8} \underbrace{101}_{\text{octal } 5} = 0101, \quad \text{hex } 7 = 0111 = 0111.$$

The second eight (**8–F**) are the octal triples with a **leading 1** — the 8’s bit turns on, and the low three bits cycle through **000–111** again:

$$\text{hex } A = \underbrace{1}_{\geq 8} \underbrace{010}_{\text{octal } 2} = 1010, \quad \text{hex } F = 1111 = 1111.$$

So the 4-bit hex table is literally the 3-bit octal table stacked twice, prefixed by **0** then **1**. The leading bit answers “is the digit ≥ 8 ?”

1.6 Remark — These Tables *Are* the Conversion Algorithm

The point of the tables is *grouping*: because $8 = 2^3$ and $16 = 2^4$, you convert by chopping a binary string into blocks and translating each block independently — no arithmetic, no carries.

- **Binary** $\rightarrow$ **octal**: group the bits into *threes* from the right, then read each triple off the octal table.
- **Binary** $\rightarrow$ **hex**: group the bits into *fours* from the right, then read each 4-tuple off the hex table.

For example, the byte 10101011_2 from Problem 6 converts both ways with zero computation:

$$\underbrace{1010}_A \underbrace{1011}_B_2 = AB_{16}, \quad \underbrace{010}_2 \underbrace{101}_5 \underbrace{011}_3_2 = 253_8.$$

(Pad on the left with zeros to fill the last group: $10101011 \rightarrow 010101011$ for octal.) Both equal **171** in decimal — a single number wearing three different base-costumes,

freely interchangeable through binary.

1.7 Remark — Why Octal and Hex, and Not (Say) Base 12

Octal and hexadecimal earn their place in computing precisely *because* 8 and 16 are powers of 2. That is what makes the per-digit grouping exact: each octal/hex digit owns a clean block of 3 or 4 bits, with no spillover between digits. A base like 12 ($\neq 2^k$) has no such tidy correspondence — converting $12 \leftrightarrow 2$ requires real division, not a table look-up. The whole convenience of this problem rests on $8 = 2^3$ and $16 = 2^4$, the same fact behind the trailing-bit tests of Problems 2 and 5 and the grouping in Problems 3 and 6. Hexadecimal won out over octal in modern practice mainly because a 4-bit group divides an 8-bit byte evenly (two hex digits per byte), whereas 3-bit octal groups straddle byte boundaries.